My Playlist DB 구축 및 쿼리 테스트

Nov 3, 2026

1. DBMS 설치

DBMS는 MySQL Community Server 를 사용합니다. 이 문서의 실행 결과는 MySQL에서 얻었습니다.

1. MySQL Installer로 MySQL Server 8.0과 MySQL Workbench를 설치합니다.
2. 설치 중에 `root` 비밀번호를 정하고 포트는 3306으로 둡니다.
3. Workbench로 접속한 뒤 `SELECT VERSION();`을 실행해 버전을 확인합니다.

2. DB와 테이블 생성

`setup.sql`을 실행하면 `my_playlist` DB와 테이블 5개, 마이그레이션 기록 테이블이 만들어집니다. 같은 파일을 두 번 실행해도 오류 없이 결과가 같았습니다. 아래는 테이블 생성 쿼리의 핵심입니다.

```
CREATE DATABASE IF NOT EXISTS my_playlist
    CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE my_playlist;

CREATE TABLE IF NOT EXISTS `USER` (
    `user_id` int NOT NULL AUTO_INCREMENT,
    `user_name` varchar(50) NOT NULL,
    `user_pw` varchar(255) NOT NULL,
    `created_at` datetime(6) NOT NULL,
    PRIMARY KEY (`user_id`),
    UNIQUE KEY `IX_USER_user_name` (`user_name`)
);

CREATE TABLE IF NOT EXISTS `PLAYLIST` (
    `playlist_id` int NOT NULL AUTO_INCREMENT,
    `user_id` int NOT NULL,
    `playlist_title` varchar(100) NOT NULL,
    `description` varchar(500) DEFAULT NULL,
    `is_public` tinyint(1) NOT NULL,
    `created_at` datetime(6) NOT NULL,
    `updated_at` datetime(6) NOT NULL,
    PRIMARY KEY (`playlist_id`),
    FOREIGN KEY (`user_id`) REFERENCES `USER` (`user_id`) ON DELETE CASCADE
);
```

```
CREATE TABLE IF NOT EXISTS `PLAYLIST_TRACK` (
    `track_id` int NOT NULL AUTO_INCREMENT,
    `playlist_id` int NOT NULL,
    `youtube_video_id` varchar(30) NOT NULL,
    `music_title` varchar(200) NOT NULL,
    `artist` varchar(200) DEFAULT NULL,
    `youtube_url` varchar(500) NOT NULL,
    `sort_order` int NOT NULL,
    `added_at` datetime(6) NOT NULL,
    PRIMARY KEY (`track_id`),
    FOREIGN KEY (`playlist_id`) REFERENCES `PLAYLIST` (`playlist_id`) ON
DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS `PLAYLIST_LIKE` (
    `like_id` int NOT NULL AUTO_INCREMENT,
    `user_id` int NOT NULL,
    `playlist_id` int NOT NULL,
    `created_at` datetime(6) NOT NULL,
    PRIMARY KEY (`like_id`),
    UNIQUE KEY (`user_id`, `playlist_id`),
    FOREIGN KEY (`playlist_id`) REFERENCES `PLAYLIST` (`playlist_id`) ON
DELETE CASCADE,
    FOREIGN KEY (`user_id`) REFERENCES `USER` (`user_id`) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS `PLAYLIST_VIEW` (
    `view_id` int NOT NULL AUTO_INCREMENT,
    `user_id` int DEFAULT NULL,
    `playlist_id` int NOT NULL,
    `viewed_at` datetime(6) NOT NULL,
    PRIMARY KEY (`view_id`),
    FOREIGN KEY (`playlist_id`) REFERENCES `PLAYLIST` (`playlist_id`) ON
DELETE CASCADE,
    FOREIGN KEY (`user_id`) REFERENCES `USER` (`user_id`) ON DELETE SET NULL
);

SHOW TABLES;
```

실행 결과입니다. Windows MySQL에서는 테이블 이름이 소문자로 표시됩니다.

```
+------------------------+
| Tables_in_my_playlist |
+------------------------+
| __efmigrationshistory |
| playlist              |
| playlist_like         |
| playlist_track        |
| playlist_view         |
| user                  |
+------------------------+
```

테이블 구조는 `DESC`로 확인했습니다.

```
DESC PLAYLIST_VIEW;
```

```
+-------------+-------------+------+-----+---------+----------------+
| Field       | Type        | Null | Key | Default | Extra          |
+-------------+-------------+------+-----+---------+----------------+
| view_id     | int         | NO   | PRI | NULL    | auto_increment |
| user_id     | int         | YES  | MUL | NULL    |                |
| playlist_id | int         | NO   | MUL | NULL    |                |
| viewed_at   | datetime(6) | NO   |     | NULL    |                |
+-------------+-------------+------+-----+---------+----------------+
```

나머지 4개 테이블의 `DESC` 쿼리는 `query_test.sql`의 Q3에 있습니다.

3. 샘플 더미 데이터 삽입

`seed.sql`을 실행하면 5개 테이블에 더미 데이터 49행이 들어갑니다. 두 번 실행해도 행 수가 그대로였습니다.

테이블	행 수	구성
`USER`	5	`dummy_` 아이디 회원. 비밀번호는 모두 `password123`
`PLAYLIST`	6	공개 5개, 비공개 1개
`PLAYLIST_TRACK`	13	다른 플레이리스트에 같은 곡 포함
`PLAYLIST_LIKE`	9	본인 플레이리스트 좋아요 포함
`PLAYLIST_VIEW`	16	비로그인 조회 7건, 같은 회원의 반복 조회 포함

회원 입력 쿼리의 예시입니다. `user_pw`에는 서버의 `PasswordHasher`로 만든 해시를 넣어, 더미 회원으로 실제 로그인할 수 있습니다.

```
INSERT IGNORE INTO `USER` (user_name, user_pw, created_at) VALUES
    ('dummy_minji', '(비밀번호 해시)', '2026-10-01 09:00:00'),
    ('dummy_junho', '(비밀번호 해시)', '2026-10-02 10:00:00'),
    ('dummy_seoyeon', '(비밀번호 해시)', '2026-10-03 11:00:00'),
    ('dummy_hyunwoo', '(비밀번호 해시)', '2026-10-04 12:00:00'),
    ('dummy_jiwoo', '(비밀번호 해시)', '2026-10-05 13:00:00');
```

`seed.sql`의 마지막 확인 쿼리 결과입니다.

```
+----------------+-----------+
| table_name     | row_count |
+----------------+-----------+
| USER           |         5 |
| PLAYLIST       |         6 |
| PLAYLIST_TRACK |        13 |
| PLAYLIST_LIKE  |         9 |
| PLAYLIST_VIEW  |        16 |
+----------------+-----------+
```

서버 API로도 확인했습니다. `dummy_minji`로 로그인하면 200이 나오고 내 플레이리스트는 3개이며, 비로그인 검색에는 비공개 플레이리스트가 나오지 않았습니다. 더미 데이터를 지우려면 다음 쿼리를 실행합니다.

```
DELETE FROM `USER` WHERE user_name LIKE 'dummy\_%';
```

4. 조회 쿼리 테스트

조회 쿼리 8개는 모두 더미 데이터의 구성과 맞는 결과를 돌려주었습니다. 번호는 `query_test.sql`의 번호와 같습니다.

Q1. DBMS 버전 확인

```
SELECT VERSION() AS mysql_version;
```

```
+---------------+
| mysql_version |
+---------------+
| 8.0.36        |
+---------------+
```

Q4. 더미 회원 조회

```
SELECT user_id, user_name, created_at
FROM `USER`
WHERE user_name LIKE 'dummy\_%'
ORDER BY user_id;
```

```
+---------+--------------+---------------------------+
| user_id | user_name    | created_at                |
+---------+--------------+---------------------------+
|       1 | dummy_minji  | 2026-10-01 09:00:00.000000 |
|       2 | dummy_junho  | 2026-10-02 10:00:00.000000 |
|       3 | dummy_seoyeon | 2026-10-03 11:00:00.000000 |
|       4 | dummy_hyunwoo | 2026-10-04 12:00:00.000000 |
|       5 | dummy_jiwoo  | 2026-10-05 13:00:00.000000 |
+---------+--------------+---------------------------+
```

Q5. 공개 플레이리스트 검색

제목에 "노래"가 들어간 플레이리스트 3개 중, 비공개인 "혼자 듣는 노래"는 나오지 않습니다.

```
SELECT p.playlist_id, p.playlist_title, u.user_name, p.is_public
FROM PLAYLIST p
JOIN `USER` u ON u.user_id = p.user_id
WHERE p.is_public = 1
  AND (p.playlist_title LIKE '%노래%' OR u.user_name LIKE '%노래%')
ORDER BY p.created_at DESC;
```

```
+-------------+-----------------------------+--------------+-----------+
| playlist_id | playlist_title              | user_name    | is_public |
+-------------+-----------------------------+--------------+-----------+
|           6 | 공부할 때 듣는 노래         | dummy_seoyeon |         1 |
|           3 | 새벽 감성 노래              | dummy_minji  |         1 |
+-------------+-----------------------------+--------------+-----------+
```

Q6. 인기 플레이리스트 순위

```
SELECT p.playlist_title, u.user_name,
    (SELECT COUNT(*) FROM PLAYLIST_LIKE l WHERE l.playlist_id =
p.playlist_id) AS like_count,
    (SELECT COUNT(*) FROM PLAYLIST_VIEW v WHERE v.playlist_id =
p.playlist_id) AS view_count
FROM PLAYLIST p
JOIN `USER` u ON u.user_id = p.user_id
WHERE p.is_public = 1
ORDER BY like_count DESC, view_count DESC, p.playlist_id DESC
LIMIT 5;
```

```
+---------------------------------+--------------+------------+------------+
| playlist_title                  | user_name    | like_count | view_count |
+---------------------------------+--------------+------------+------------+
| 새벽 감성 노래                  | dummy_minji  |          5 |          6 |
| 드라이브 플레이리스트           | dummy_junho  |          2 |          4 |
| 운동할 때 듣는 팝               | dummy_minji  |          1 |          3 |
| 공부할 때 듣는 노래             | dummy_seoyeon |          1 |          1 |
| 추억의 팝송                     | dummy_hyunwoo |          0 |          2 |
+---------------------------------+--------------+------------+------------+
```

Q7. 플레이리스트의 곡 목록

```
SELECT t.sort_order, t.music_title, t.artist, t.youtube_url
FROM PLAYLIST_TRACK t
JOIN PLAYLIST p ON p.playlist_id = t.playlist_id
WHERE p.playlist_title = '새벽 감성 노래'
ORDER BY t.sort_order;
```

```
+------------+-------------+------------+-------------------------------------
---------+
| sort_order | music_title | artist     | youtube_url
|
+------------+-------------+------------+-------------------------------------
---------+
|          1 | 밤편지      | 아이유     | https://www.youtube.com/watch?
v=BzYnNdJhZQw |
|          2 | Perfect     | Ed Sheeran | https://www.youtube.com/watch?
v=2Vv-BfVoq4g |
|          3 | Hello       | Adele      | https://www.youtube.com/watch?
v=YQHsXMglC9A |
+------------+-------------+------------+-------------------------------------
---------+
```

Q8. 회원이 좋아요한 플레이리스트

```
SELECT p.playlist_title, owner_user.user_name AS owner_name, l.created_at AS
liked_at
FROM PLAYLIST_LIKE l
JOIN `USER` u ON u.user_id = l.user_id
JOIN PLAYLIST p ON p.playlist_id = l.playlist_id
JOIN `USER` owner_user ON owner_user.user_id = p.user_id
WHERE u.user_name = 'dummy_jiwoo'
ORDER BY l.like_id DESC;
```

```
+---------------------------------+--------------+--------------------------
--+
| playlist_title                  | owner_name    | liked_at
|
+---------------------------------+--------------+--------------------------
--+
| 공부할 때 듣는 노래             | dummy_seoyeon | 2026-10-17 12:00:00.000000 |
| 새벽 감성 노래                  | dummy_minji   | 2026-10-16 13:00:00.000000 |
| 드라이브 플레이리스트           | dummy_junho   | 2026-10-17 10:00:00.000000 |
+---------------------------------+--------------+--------------------------
--+
```

Q9. 조회 기록 집계

회원 조회와 비로그인 조회를 나누어 셉니다.

```
SELECT p.playlist_title,
    COUNT(*) AS total_views,
    COUNT(v.user_id) AS member_views,
    SUM(v.user_id IS NULL) AS anonymous_views
FROM PLAYLIST_VIEW v
JOIN PLAYLIST p ON p.playlist_id = v.playlist_id
GROUP BY p.playlist_id, p.playlist_title
ORDER BY total_views DESC, p.playlist_id;
```

```
+--------------------------------+-------------+--------------+-------------
----+
| playlist_title                 | total_views | member_views |
anonymous_views |
+--------------------------------+-------------+--------------+-------------
----+
| 새벽 감성 노래                 |           6 |            4 |               2
|
| 드라이브 플레이리스트          |           4 |            2 |               2
|
| 운동할 때 듣는 팝              |           3 |            2 |               1
|
| 추억의 팝송                    |           2 |            0 |
2 |
| 공부할 때 듣는 노래            |           1 |            1 |               0
|
+--------------------------------+-------------+--------------+-------------
----+
```

Q10. 전체 통계

```
SELECT
    (SELECT COUNT(*) FROM `USER`) AS user_count,
    (SELECT COUNT(*) FROM PLAYLIST) AS playlist_count,
    (SELECT COUNT(*) FROM PLAYLIST_TRACK) AS track_count,
    (SELECT COUNT(*) FROM PLAYLIST_LIKE) AS like_count,
    (SELECT COUNT(*) FROM PLAYLIST_VIEW) AS view_count;
```

```
+------------+----------------+-------------+------------+------------+
| user_count | playlist_count | track_count | like_count | view_count |
+------------+----------------+-------------+------------+------------+
|          5 |              6 |          13 |          9 |         16 |
+------------+----------------+-------------+------------+------------+
```

5. 수정, 삭제, 제약조건 테스트

수정, 연쇄 삭제, 중복 차단이 모두 설계대로 동작했습니다. Q11~Q13은 `ROLLBACK`으로 되돌려, 실행 후에도 행 수가 5, 6, 13, 9, 16으로 그대로였습니다.

Q11. 플레이리스트 수정

```
START TRANSACTION;
UPDATE PLAYLIST
SET playlist_title = '새벽 감성 노래 (수정)', is_public = 0
WHERE playlist_title = '새벽 감성 노래';
SELECT playlist_title, is_public
FROM PLAYLIST
WHERE playlist_title LIKE '새벽 감성 노래%';
ROLLBACK;
```

```
+--------------------------------+-----------+
| playlist_title                 | is_public |
+--------------------------------+-----------+
| 새벽 감성 노래 (수정)          |         0 |
+--------------------------------+-----------+
```

Q12. 플레이리스트 삭제

플레이리스트를 지우면 곡, 좋아요, 조회 기록이 함께 삭제됩니다.

```
START TRANSACTION;
SET @target = (SELECT playlist_id FROM PLAYLIST WHERE playlist_title = '새벽
감성 노래');
SELECT 'before' AS step,
    (SELECT COUNT(*) FROM PLAYLIST_TRACK WHERE playlist_id = @target) AS
tracks,
    (SELECT COUNT(*) FROM PLAYLIST_LIKE WHERE playlist_id = @target) AS
likes,
    (SELECT COUNT(*) FROM PLAYLIST_VIEW WHERE playlist_id = @target) AS
views;
DELETE FROM PLAYLIST WHERE playlist_id = @target;
SELECT 'after' AS step,
    (SELECT COUNT(*) FROM PLAYLIST_TRACK WHERE playlist_id = @target) AS
tracks,
    (SELECT COUNT(*) FROM PLAYLIST_LIKE WHERE playlist_id = @target) AS
likes,
    (SELECT COUNT(*) FROM PLAYLIST_VIEW WHERE playlist_id = @target) AS
views;
ROLLBACK;
```

```
+--------+--------+-------+-------+
| step   | tracks | likes | views |
+--------+--------+-------+-------+
| before |      3 |     5 |     6 |
+--------+--------+-------+-------+
+-------+--------+-------+-------+
| step  | tracks | likes | views |
+-------+--------+-------+-------+
| after |      0 |     0 |     0 |
+-------+--------+-------+-------+
```

Q13. 회원탈퇴

회원의 좋아요는 삭제되고, 조회 기록은 `user_id`만 NULL이 되어 전체 조회수 16이 유지됩니다.

```
START TRANSACTION;
SET @target_user = (SELECT user_id FROM `USER` WHERE user_name =
'dummy_jiwoo');
SET @member_views = (SELECT COUNT(*) FROM PLAYLIST_VIEW WHERE user_id =
@target_user);
SET @total_views = (SELECT COUNT(*) FROM PLAYLIST_VIEW);
SELECT 'before' AS step,
    (SELECT COUNT(*) FROM PLAYLIST_LIKE WHERE user_id = @target_user) AS
user_likes,
    @member_views AS user_views,
    @total_views AS total_views;
DELETE FROM `USER` WHERE user_id = @target_user;
SELECT 'after' AS step,
    (SELECT COUNT(*) FROM PLAYLIST_LIKE WHERE user_id = @target_user) AS
user_likes,
    (SELECT COUNT(*) FROM PLAYLIST_VIEW WHERE user_id = @target_user) AS
user_views,
    (SELECT COUNT(*) FROM PLAYLIST_VIEW) AS total_views;
ROLLBACK;
```

```
+--------+------------+------------+-------------+
| step   | user_likes | user_views | total_views |
+--------+------------+------------+-------------+
| before |          3 |          3 |          16 |
+--------+------------+------------+-------------+
+-------+------------+------------+-------------+
| step  | user_likes | user_views | total_views |
+-------+------------+------------+-------------+
| after |          0 |          0 |          16 |
+-------+------------+------------+-------------+
```

Q14. 중복 아이디 입력 차단

`ERROR 1062`는 UNIQUE 제약조건이 중복을 막았다는 뜻이므로 정상 결과입니다.

```
INSERT INTO `USER` (user_name, user_pw, created_at)
VALUES ('dummy_minji', 'x', NOW());
```

```
ERROR 1062 (23000) at line 121: Duplicate entry 'dummy_minji' for key
'user.IX_USER_user_name'
```

Q15. 중복 좋아요 입력 차단

```
INSERT INTO PLAYLIST_LIKE (user_id, playlist_id, created_at)
SELECT l.user_id, l.playlist_id, NOW()
FROM PLAYLIST_LIKE l
JOIN `USER` u ON u.user_id = l.user_id
WHERE u.user_name = 'dummy_jiwoo'
LIMIT 1;
```

```
ERROR 1062 (23000) at line 125: Duplicate entry '5-2' for key
'playlist_like.IX_PLAYLIST_LIKE_user_id_playlist_id'
```

6. 실행 결과 화면 캡처

위 결과는 빈 임시 DB에서 `mysql` 명령줄 클라이언트로 실행한 실제 출력이며, Workbench 화면 캡처는 아직 넣지 않았습니다. 제출에 화면 캡처가 필요하면 아래 순서로 찍어 각 쿼리 아래에 붙입니다.

1. Workbench에서 `setup.sql`을 열어 `Ctrl + Shift + Enter`로 전체 실행하고 결과 화면을 캡처합니다.
2. `seed.sql`을 같은 방법으로 실행하고 결과 화면을 캡처합니다.
3. `query_test.sql`을 열고, 쿼리를 Q번호별로 드래그해 선택한 뒤 `Ctrl + Shift + Enter`로 실행합니다.
4. 쿼리와 Result Grid가 함께 보이게 캡처합니다. Q14, Q15는 아래 Output 창의 오류 메시지가 보이게 캡처합니다.

이미 데이터가 있는 DB에서는 번호와 Q10의 행 수가 위 결과와 다를 수 있습니다. Q5~Q9, Q11~Q13의 일부 결과도 기존 데이터의 영향을 받을 수 있으므로, 결과를 똑같이 맞추려면 빈 DB에서 실행합니다.